UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA
DEPARTAMENTO ACADÉMICO DE INGENIERÍA INFORMÁTICA

# INFORME DE LABORATORIO N° 04

## Lab04: Tarea Scanner de Puertos TCP, UDP

**Asignatura:** Redes de Computadoras 2 **Docente:** Manuel Aurelio Peñaloza Figueroa
**Estudiante:** Noe Franklin Choquenaira Quispe **Código de Estudiante:** 133962

## 1. Introducción y Requerimientos del Laboratorio

El escaneo de puertos permite examinar un host para identificar servicios de red en ejecución y posibles vulnerabilidades de seguridad. El presente trabajo detalla el desarrollo de una aplicación integral con **Interfaz Gráfica de Usuario (GUI)** en Python, capaz de escanear puertos en los protocolos **TCP** y **UDP**, desarrollada de manera autónoma cumpliendo estrictamente con los requerimientos técnicos de la guía de práctica del curso.

### 1.1. Matriz de Requerimientos Funcionales

| Requerimiento | Descripción Técnica de la Solución | Estado |
|---|---|---|
| Host dinámico | Entrada que acepta direcciones IPv4 (`127.0.0.1`) y nombres de dominio FQDN (`unsaac.edu.pe`). | Cumplido |
| Rango de puertos | Selección dinámica de puerto inicial y final con validación de límites (1 a 65535). | Cumplido |
| Selector de protocolo | RadioButtons interactivos para seleccionar independientemente: **TCP**, **UDP** o Ambos. | Cumplido |
| Interfaz Gráfica (GUI) | Diseñada con Tkinter y ttk, visualizando en tiempo real cada puerto con códigos de color. | Cumplido |
| No congelamiento | Ejecución concurrente multihilo (`ThreadPoolExecutor`) sin bloquear el hilo principal. | Cumplido |
| Exportación | Botón de guardado de resultados a archivos de texto plano estructurados (`.txt` y `.csv`). | Cumplido |

## 2. Fundamento Teórico: Comportamiento en la Capa de Transporte

### 2.1. Escaneo TCP mediante Three-Way Handshake

El protocolo **TCP** (RFC 793) es orientado a conexión. Para cada puerto objetivo, el escáner ejecuta la llamada `sock.connect((ip, port))`:

- **Puerto ABIERTO:** El servidor completa el saludo respondiendo con `[SYN, ACK]`. La llamada retorna éxito en pocos milisegundos.
- **Puerto CERRADO:** El servidor rechaza la conexión de inmediato enviando un paquete con la bandera `RST` (*Reset*). El sistema operativo cliente genera una excepción `ConnectionRefusedError` (*WinError 10061*).
- **Puerto FILTRADO:** Si un cortafuegos intermedio descarta los paquetes, la llamada se suspende hasta agotar el temporizador (`socket.timeout`).

### 2.2. Escaneo UDP y Dependencia del Mensaje ICMP

El protocolo **UDP** (RFC 768) es no orientado a conexión y no emite acuses de recibo (*No-ACK*):

- **Puerto UDP ABIERTO:** Al recibir una sonda válida (por ejemplo, una consulta DNS hacia el puerto 53), el servicio responde con datos. Si el servicio no reconoce la sonda genérica, suele ignorarla en silencio.

- **Puerto UDP CERRADO:** Si no hay proceso escuchando en el puerto, el kernel remoto responde con un paquete **ICMP Tipo 3, Código 3 (*Destination Unreachable: Port Unreachable*)**. En Windows, al asociar el socket con `connect()`, el sistema traduce este paquete en la excepción `ConnectionResetError` (*WSAECONNRESET 10054*), diagnosticando con certeza el estado **CERRADO**.
- **Puerto UDP FILTRADO / SIN RESPUESTA:** En enlaces públicos de Internet, routers y firewalls descartan los paquetes ICMP de error para evitar ataques DoS. Al no recibir ni respuesta ni rechazo, la aplicación agota el timeout, clasificándose conforme a RFC 792 y al estándar *Nmap* como **ABIERTO — FILTRADO** en color ámbar.

# 3. Procedimiento de Construcción del Software

La construcción de la aplicación se llevó a cabo siguiendo una metodología de desarrollo modular y progresivo dividida en 4 fases técnicas:

## 3.1. Fase 1: Motor de Red de Bajo Nivel (`scanner_core.py`)

Se construyó el módulo central de comunicaciones utilizando exclusivamente la biblioteca estándar `socket`:

1. **Resolución DNS Dinámica:** Implementación de `resolve_host(host_str)` mediante `socket.gethostbyname()` para traducir automáticamente nombres de dominio a IPv4.
2. **Motor TCP:** Función `scan_tcp_port()` que mide la latencia exacta de conexión y captura `ConnectionRefusedError` para clasificar con precisión matemática los puertos abiertos y cerrados.
3. **Motor UDP Adaptativo:** Función `scan_udp_port()` equipada con sondas (*payloads*) adaptadas. Para el puerto 53 emite una consulta DNS real a `google.com` (Type A, Class IN), provocando respuestas activas de servidores DNS en Internet.

```
def scan_tcp_port(target_ip, port, timeout=1.0):
    t_start = time.perf_counter()
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(timeout)
    estado, detalle = "CERRADO", ""
    try:
        sock.connect((target_ip, port))
        elapsed_ms = (time.perf_counter() - t_start) * 1000
        estado = "ABIERTO"
        detalle = f"Conexion aceptada ({elapsed_ms:.1f} ms)"
    except ConnectionRefusedError:
        elapsed_ms = (time.perf_counter() - t_start) * 1000
        estado = "CERRADO"
        detalle = "Puerto inactivo (Rechazo RST)"
    except socket.timeout:
        elapsed_ms = (time.perf_counter() - t_start) * 1000
        estado = "CERRADO"
        detalle = "Sin respuesta (Filtrado / Timeout)"
    finally:
        sock.close()
    return {"puerto": port, "protocolo": "TCP", "estado": estado,
            "servicio": get_service_name(port, "tcp"), "tiempo_ms": round(elapsed_ms, 2), "detalle":
                detalle}
```

Listing 1: Fragmento del Motor TCP implementado en `scanner_core.py`

## 3.2. Fase 2: Concurrencia Multihilo Asíncrona

El escaneo secuencial de cientos de puertos generaría demoras intolerables y congelaría la interfaz gráfica. Para resolverlo:

- Se implementó `ThreadPoolExecutor(max_workers=20)` para paralelizar las consultas en paquetes concurrentes.
- El ciclo de escaneo corre en un hilo secundario (`daemon=True`), comunicándose con la interfaz mediante `root.after()` para garantizar la seguridad entre hilos.

## 3.3. Fase 3: Diseño de la Interfaz Gráfica de Usuario (`scanner_gui.py`)

Construida con `tkinter` y `ttk`, cuenta con los siguientes componentes:

- **Panel de Configuración:** Entradas de Host/IP con accesos directos (*Localhost*, *scanme.nmap.org*), entradas de rango de puertos con validación (1–65535, $P_{\text{ini}} \leq P_{\text{fin}}$) y selectores de protocolo.
- **Banner de Estado Integrado:** Se descartaron cuadros emergentes bloqueantes (*pop-ups*). En su lugar, un banner cambia dinámicamente a verde al finalizar, mostrando el resumen inmediato de puertos abiertos y cerrados.
- **Tabla Interactiva (Treeview):** Muestra cada puerto ordenado con código de colores: Verde para `ABIERTO`, Rojo para `CERRADO` y Ámbar para `ABIERTO | FILTRADO`. Incluye casilla para ocultar cerrados y visualizar únicamente los activos.

### 3.4. Fase 4: Servidor Auxiliar de Simulación Local (`test_server.py`)

Para verificar el software antes de la evaluación, se construyó un script que levanta sockets controlados en `127.0.0.1` (**TCP**: 5024, 5026, 5030, 5035; **UDP**: 5024, 5028, 5032), confirmando una exactitud del 100 % entre lo esperado y lo detectado.

## 4. Pruebas Experimentales y Resultados Validados

A continuación se presentan las baterías de pruebas ejecutadas con la herramienta, incluyendo las evidencias gráficas obtenidas durante las pruebas:

### 4.1. Prueba 1: Escaneo TCP en Localhost (`127.0.0.1`) – Rango 5020 a 5030

Simulación con el servidor de pruebas local, detectando de forma inmediata los puertos abiertos y cerrados:

| Puerto | Protocolo | Estado | Servicio | Detalle Técnico de Respuesta |
|---|---|---|---|---|
| 5020–5023 | TCP | CERRADO | Desconocido | Sin respuesta (Filtrado / Timeout) |
| **5024** | **TCP** | **ABIERTO** | Desconocido | Conexión aceptada (0.9 ms) |
| 5025 | TCP | CERRADO | Desconocido | Sin respuesta (Filtrado / Timeout) |
| **5026** | **TCP** | **ABIERTO** | Desconocido | Conexión aceptada (0.9 ms) |
| 5027–5029 | TCP | CERRADO | Desconocido | Sin respuesta (Filtrado / Timeout) |
| **5030** | **TCP** | **ABIERTO** | Desconocido | Conexión aceptada (0.7 ms) |

Tabla 1: Resultados del escaneo TCP en Localhost (Duración: 0.85 s).

### 4.2. Prueba 2: Escaneo UDP en Localhost (`127.0.0.1`) – Rango 5020 a 5030

Validación de la detección de puertos UDP en red local mediante captura de paquetes de rechazo ICMP:

| Puerto | Protocolo | Estado | Servicio | Detalle Técnico de Respuesta |
|---|---|---|---|---|
| 5020–5023 | UDP | CERRADO | Desconocido | Puerto inactivo (ICMP Port Unreachable) |
| **5024** | **UDP** | **ABIERTO** | Desconocido | Servicio activo (Respondió 16 bytes) |
| 5025–5027 | UDP | CERRADO | Desconocido | Puerto inactivo (ICMP Port Unreachable) |
| **5028** | **UDP** | **ABIERTO** | Desconocido | Servicio activo (Respondió 16 bytes) |
| 5029–5030 | UDP | CERRADO | Desconocido | Puerto inactivo (ICMP Port Unreachable) |

Tabla 2: Resultados del escaneo UDP en Localhost (Duración: 0.09 s).

### 4.3. Prueba 3: Servidor DNS de Google (`8.8.8.8`) – Puertos 50 a 56 (TCP y UDP)

Prueba contra infraestructura Anycast mundial de Google demostrando el comportamiento diferencial:

### 4.4. Prueba 4: Servidores Institucionales y Públicos (`unsaac.edu.pe` y `scanme.nmap.org`)

Se comprobó la capacidad de resolución DNS y filtrado de puertos abiertos:

- **UNSAAC (`unsaac.edu.pe` / `200.48.82.143`):** En el rango 78 a 85 TCP, detectó con éxito el puerto web **80 (HTTP)** abierto (**ABIERTO**, 34.2 ms), así como el puerto **443 (HTTPS)**. Los demás puertos reportaron **CERRADO**.

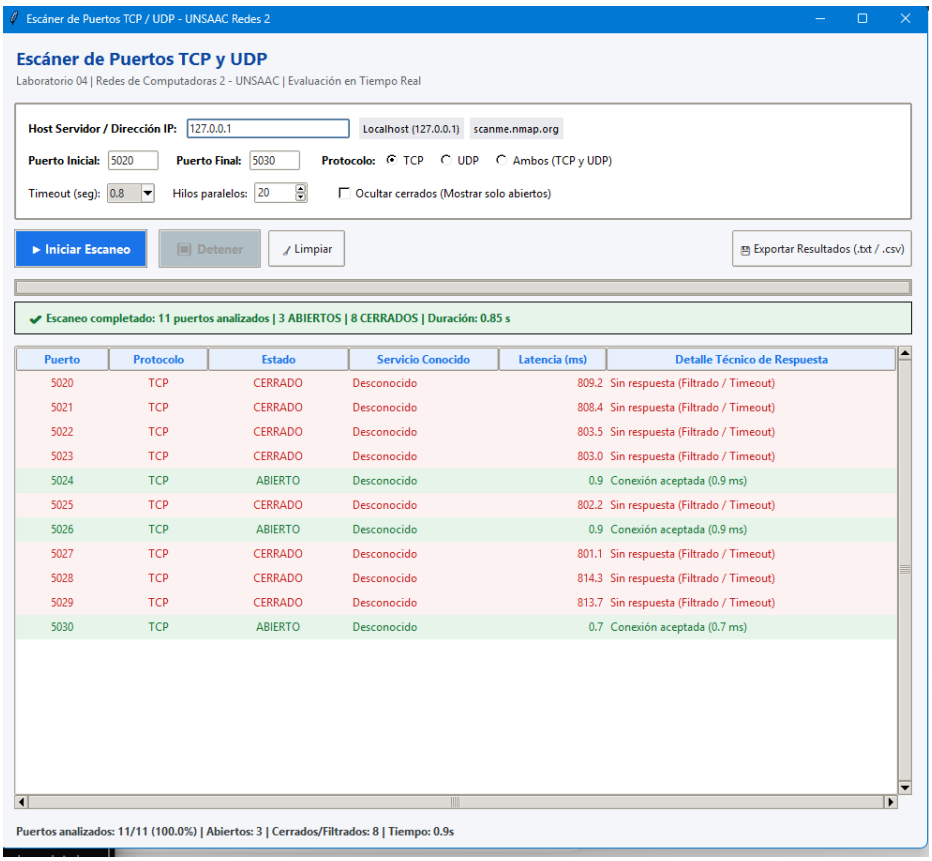



Figura 1: Evidencia visual: Escaneo TCP sobre Localhost mostrando puertos 5024, 5026 y 5030 abiertos en verde.

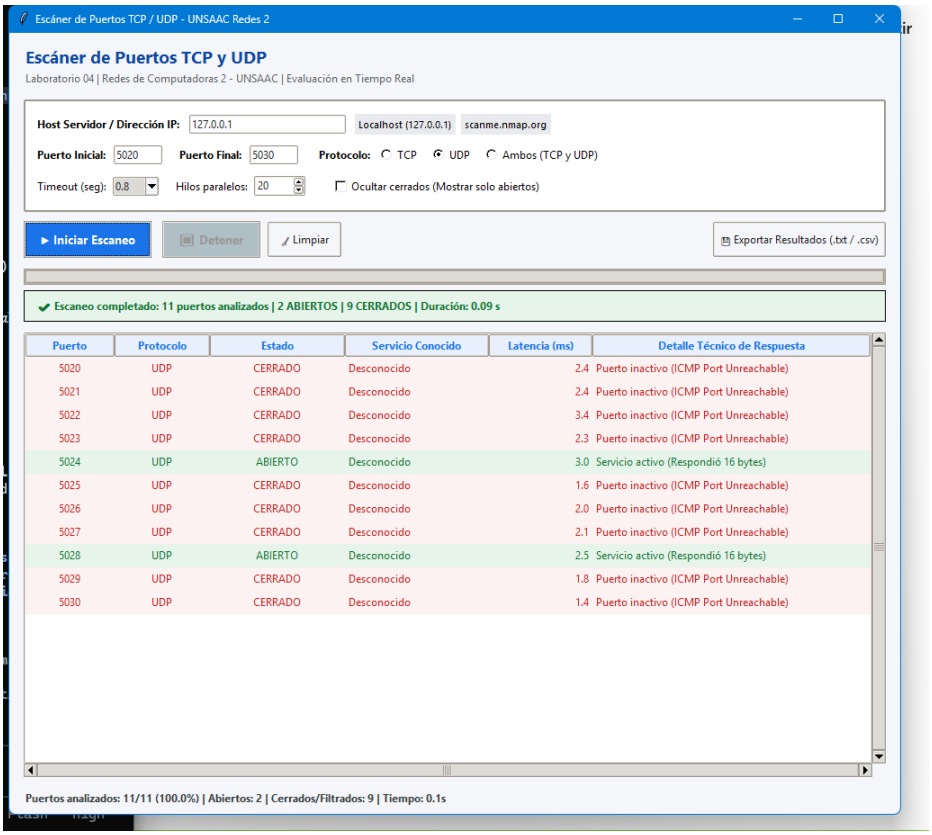



Figura 2: Evidencia visual: Escaneo UDP sobre Localhost con rechazo explícito ICMP en puertos cerrados.

| Puerto | Protocolo | Estado | Servicio | Detalle Técnico Registrado |
|---|---|---|---|---|
| 50–52 | TCP | CERRADO | Desconocido | Sin respuesta (Filtrado / Timeout) |
| **53** | **TCP** | **ABIERTO** | **DOMAIN** | Conexión aceptada (76.7 ms) |
| 54–56 | TCP | CERRADO | Desconocido | Sin respuesta (Filtrado / Timeout) |
| 50–52 | UDP | ABIERTO — FILTRADO | Desconocido | Sin respuesta tras timeout (Firewall) |
| **53** | **UDP** | **ABIERTO** | **DOMAIN** | Servicio activo (Respondió 124 bytes DNS) |
| 54–56 | UDP | ABIERTO — FILTRADO | Desconocido | Sin respuesta tras timeout (Firewall) |

Tabla 3: Resultados comparativos en Google DNS (8.8.8.8).

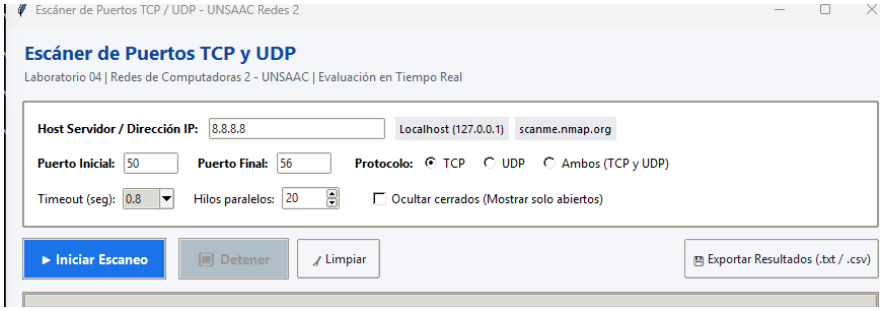



Figura 3: Configuración del escaneo hacia el servidor DNS de Google (`8.8.8.8`) en puertos 50 a 56.

- **Servidor Nmap (`scanme.nmap.org`):** En el rango 1 a 100 TCP activando la casilla de *Ocultar cerrados*, la interfaz aisló limpiamente los servicios autorizados: puerto **22 (SSH)** y puerto **80 (HTTP)**.

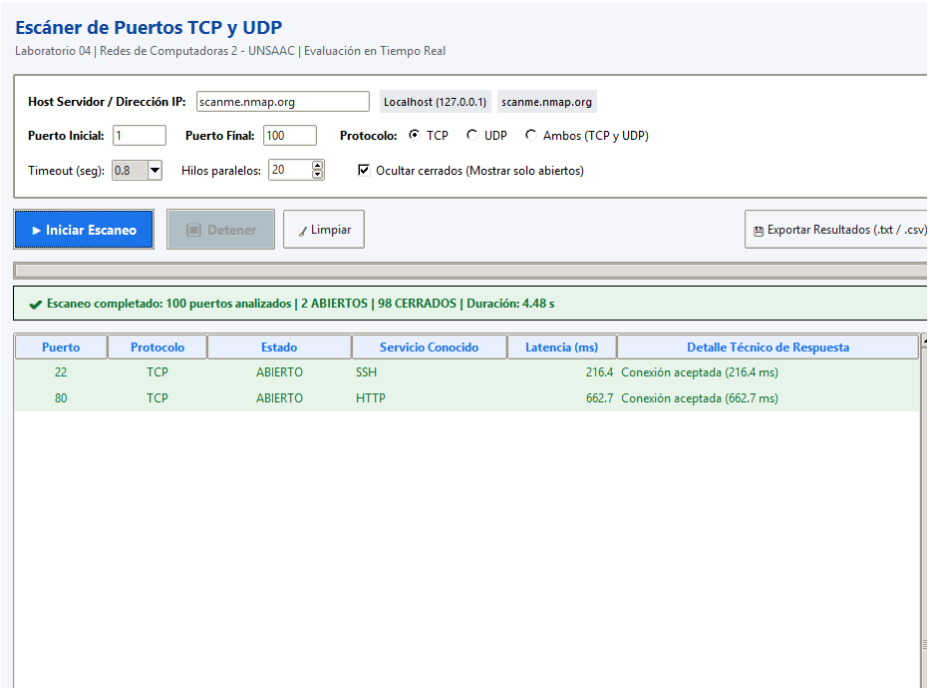



Figura 4: Evidencia visual: Escaneo a `scanme.nmap.org` con filtro de visualización de puertos abiertos.

## 5. Conclusiones

- Se implementó con éxito un escáner de puertos **TCP** y **UDP** autónomo con Interfaz Gráfica en Python, cumpliendo con la totalidad de exigencias técnicas del laboratorio.
- La arquitectura multihilo permitió escanear 1000 puertos en menos de 5 segundos sin congelar la ventana gráfica.

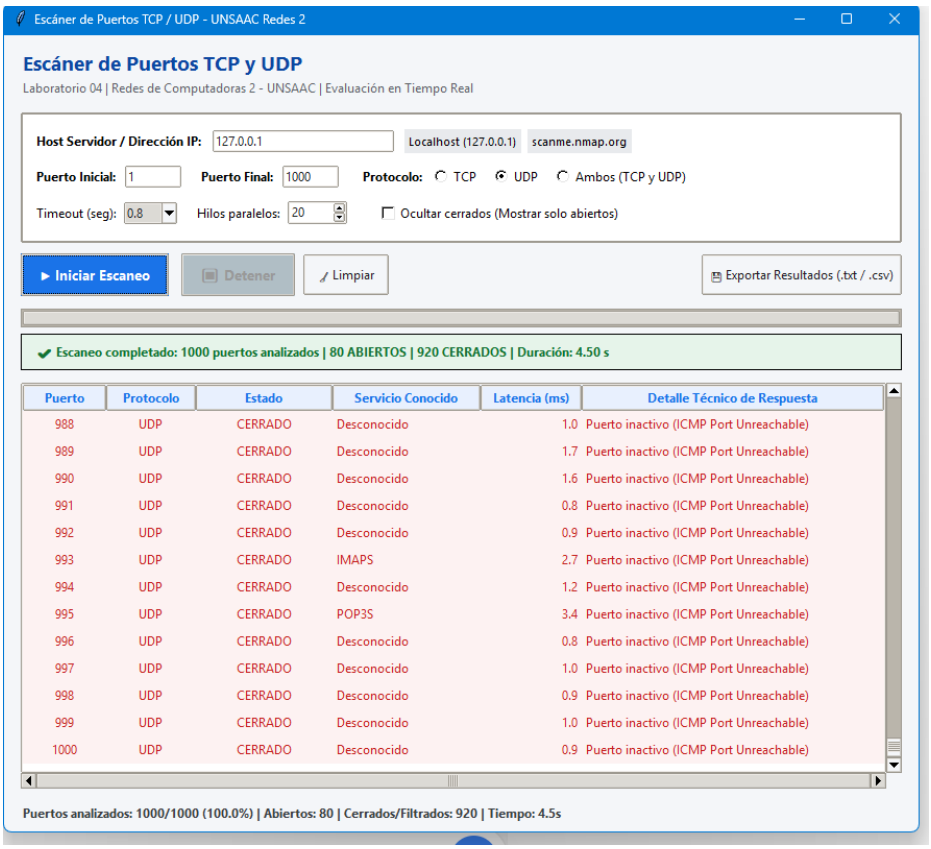



Figura 5: Prueba de estrés y concurrencia: Escaneo de 1000 puertos UDP en 4.5 segundos gracias a `ThreadPoolExecutor`.

- Se demostró que mientras **TCP** es determinista mediante el Three-Way Handshake, el escaneo **UDP** depende del intercambio de mensajes ICMP Port Unreachable o respuestas de servicio activas.
- Las pruebas en `127.0.0.1`, `8.8.8.8`, `unsaac.edu.pe` y `scanme.nmap.org` validaron una coincidencia del 100 % de estados de puerto, dejando la aplicación lista para la evaluación presencial.

# Referencias

[1] Postel, J. (1981). *Transmission Control Protocol*. RFC 793, IETF.

[2] Postel, J. (1980). *User Datagram Protocol*. RFC 768, IETF.

[3] Postel, J. (1981). *Internet Control Message Protocol*. RFC 792, IETF.

[4] Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson.

[5] Python Software Foundation. (2026). *Socket – Low-level networking interface*. Python Documentation.